

Clase 0

Configuración, cuentas y OpenCode

Guía de trabajo del aprendiz · Sesión de preparación

Sesión rápida de preparación (no es una clase con horario)

Pablo Andrés Carvajal — Desarrollador Full Stack

pablocarvajal.dev

Nombre del aprendiz: _____



¿Qué sabré hacer al terminar esta guía?

- Explicar qué es una API y una API key, y por qué la clave es secreta.
- Crear cuentas y obtener claves en Google AI Studio (Gemini), Cohere y GitHub.
- Instalar Python, VS Code y OpenCode.
- Guardar mis claves de forma segura en un archivo `.env`.
- Hacer mi primera petición a un modelo de IA desde Python.

TEN EN CUENTA

La Clase 0 **no es una clase con horario**: es una preparación rápida para dejar tu entorno listo. Puedes completarla en casa siguiendo esta guía paso a paso, para que en la Clase 1 empecemos directamente con el contenido.

Marca cada casilla a medida que avanzas.

1. Conceptos base

¿Qué es una API?

Piensa en un **restaurante**: el **modelo de IA** es la cocina, tú eres el cliente y la **API** es el mesero que lleva tu pedido y te trae la respuesta. No necesitas entrar a la cocina.

Una API es, entonces, la forma en que un programa le pide algo a otro. Nosotros le pedimos respuestas a la IA **sin tenerla instalada** en el computador.

¿Qué es una API key?

Es una clave personal que **identifica quién hace el pedido**, para medir y limitar el uso (como una tarjeta de cliente).

REGLA DE ORO

La API key es **personal y secreta**. Nunca se comparte, nunca se sube a internet y nunca se pega en un chat público. Si se filtra, otra persona puede usar tu cuenta.

Tres palabras que vas a oír: SDK, librería y repositorio

Durante el curso instalaremos "librerías", usaremos "SDKs" y trabajaremos con "repos". Conviene tener clara la diferencia desde el principio:

- **Librería (library):** un conjunto de código ya hecho para **una tarea concreta**. La importas en tu programa y la usas. Ejemplos: `numpy` (matemáticas), `requests` (conectarse a internet).
- **SDK (Software Development Kit, "kit de desarrollo"):** un **kit completo** para trabajar con una plataforma o servicio. Suele incluir **una o varias librerías** más herramientas, ejemplos y documentación, y normalmente es **oficial** del proveedor. Ejemplos: el SDK de Gemini (`google-generativeai`) o el de Cohere (`cohere`).
- **Repositorio (repo):** el **lugar donde se guarda y se versiona el código** (con Git, por ejemplo en GitHub). No es un tipo de código, sino su "almacén" con historial de cambios. Dentro de un repo puede haber una librería, un SDK o una aplicación entera.

Concepto	Qué es	Analogía (taller)
Librería	Código para una tarea	Una herramienta (un destornillador)
SDK	Kit para una plataforma (incluye librerías)	Una caja de herramientas con manual
Repo	Dónde se guarda y versiona el código	El estante/bodega con su historial

EN RESUMEN

Un **SDK** suele contener **librerías**; un **repositorio** es donde vive el código de cualquiera de los dos.

2. Crear cuentas y obtener claves

2.1 Google AI Studio (Gemini) — gratis

1. Entra a `aistudio.google.com` e inicia sesión con tu cuenta de Google.
2. Acepta los términos.
3. Haz clic en **"Get API key" → "Create API key"**.
4. Copia la clave y guárdala temporalmente en un bloc de notas.

2.2 Cohere — gratis

1. Entra a `dashboard.cohere.com` y regístrate.
2. Ve a la sección **"API Keys"**.
3. Copia la **Trial key** (gratuita, con límites de uso).

2.3 GitHub — gratis

1. Entra a `github.com` y haz clic en **Sign up**.

2. Crea tu usuario y verifica el correo.

2.4 Guardar las claves de forma segura

Crea un archivo llamado `.env` con este contenido (reemplazando por tus claves):

```
GEMINI_API_KEY=pega_aqui_tu_clave_de_gemini  
COHERE_API_KEY=pega_aqui_tu_clave_de_cohere
```

RECUERDA

El `.env` guarda secretos y nunca se sube a GitHub.

3. Instalar las herramientas

3.1 Python

- Descárgalo desde python.org/downloads (versión 3.11 o superior).
- **En Windows marca la casilla "Add Python to PATH".**
- Verifica con: `python --version`

3.2 VS Code

- Descárgalo desde code.visualstudio.com.
- Instala las extensiones **Python** y **Jupyter**.

4. OpenCode a fondo

OpenCode es un agente de IA de código abierto que trabaja en la **terminal**: le escribes en lenguaje natural y él lee, crea y modifica archivos por ti, ejecuta comandos y construye aplicaciones completas. Funciona con APIs gratuitas (Gemini, Cohere, GitHub Models) y también con modelos de pago.

¿POR QUÉ NOS INTERESA EN ESTE CURSO?

OpenCode es un **agente de IA real** (un modelo + herramientas + memoria + capacidad de actuar). Usarlo desde ahora nos ayuda a entender, en la práctica, lo que construiremos nosotros mismos en la Clase 4.

4.1 Instalación y la TUI

```
npm install -g opencode-ai
opencode --version
```

Si no tienes Node.js, instálalo desde nodejs.org (versión LTS). Luego, dentro de una carpeta de proyecto, ejecuta `opencode`, elige proveedor (Gemini/Cohere), pega tu API key y elige un modelo gratuito.

La **TUI** (interfaz de texto) se maneja sin mouse. Dentro de ella escribes tu petición en lenguaje natural y OpenCode actúa. Hay tres formas de "hablarle": `/` para comandos, `@` para mencionar archivos y `!` para ejecutar comandos del sistema (lo vemos abajo).

Comandos con `/`

Escribe `/` y aparece la lista. Los más importantes:

Comando	Para qué sirve
<code>/help</code>	Ver la ayuda y los comandos disponibles
<code>/new</code> (o <code>/clear</code>)	Empezar una sesión nueva (limpia el contexto)
<code>/sessions</code>	Listar y cambiar entre sesiones
<code>/undo</code> · <code>/redo</code>	Deshacer (revierte cambios en archivos) y rehacer
<code>/models</code>	Elegir el modelo a usar
<code>/compact</code>	Resumir la sesión para gastar menos tokens
<code>/init</code>	Crear el archivo <code>AGENTS.md</code> (reglas del proyecto)
<code>/editor</code>	Redactar el mensaje en un editor externo
<code>/export</code>	Exportar la conversación a Markdown
<code>/share</code> · <code>/unshare</code>	Compartir / dejar de compartir la sesión
<code>/exit</code> (o <code>/q</code>)	Salir de OpenCode

Atajos de teclado (keybinds)

OpenCode usa una **tecla líder** (*leader*): `Ctrl+X`. Se pulsa `Ctrl+X` y **luego** otra tecla (tienes unos 2 segundos). Por ejemplo, "`Ctrl+X` `n`" significa: pulsa Ctrl+X, suéltalo y pulsa la N.

Atajo	Acción
<code>Tab</code> / <code>Shift+Tab</code>	Cambiar de modo/agente (Build ⇌ Plan)
<code>Enter</code>	Enviar el mensaje
<code>Shift+Enter</code>	Salto de línea sin enviar
<code>Esc</code>	Interrumpir / detener a la IA
<code>Ctrl+P</code>	Abrir la paleta de comandos
<code>Ctrl+X</code> <code>n</code>	Nueva sesión
<code>Ctrl+X</code> <code>l</code>	Listar / cambiar de sesión
<code>Ctrl+X</code> <code>m</code>	Elegir modelo
<code>Ctrl+X</code> <code>a</code>	Listar modos/agentes
<code>Ctrl+X</code> <code>t</code>	Cambiar el tema
<code>Ctrl+X</code> <code>e</code>	Abrir editor externo
<code>Ctrl+C</code>	Salir de OpenCode

El `@` (archivos) y el `!` (terminal)

`@` — **mentonar archivos:** escribe `@` y busca un archivo del proyecto para **dárselo como contexto** a la IA, sin copiar y pegar. Puedes mencionar varios en un mismo mensaje:

```
Explicame qué hace @app.py y corrige el error de @utils/db.py
```

`!` — **ejecutar en la terminal:** empieza el mensaje con `!` para correr un comando del sistema; su salida se incorpora a la conversación como resultado:

```
!python app.py
!git status
!pip install requests
```

REGLA RÁPIDA

`/` = comando de OpenCode · `@` = "mira este archivo" · `!` = "ejecuta esto en la terminal".

4.2 Modos Build y Plan

- **Build** (por defecto): tiene **todas las herramientas activas**; crea y modifica archivos y ejecuta comandos. Es el modo normal de trabajo.
- **Plan**: modo restringido. Antes de **editar archivos o ejecutar comandos** siempre **te pregunta**. Sirve para que la IA analice y proponga un plan sin cambiar nada.

Cómo cambiar de modo: pulsa `Tab` para alternar Build ⇌ Plan (`Shift+Tab` para volver). Con `Ctrl+X` luego `a` abres la lista de modos/agentes.

BUENA PRÁCTICA

Usa primero **Plan** para acordar el enfoque y revisar los pasos; luego cambia a **Build** para que lo ejecute.

4.3 Modo Shell y el historial

El **modo Shell** te deja ejecutar comandos de terminal (instalar dependencias, correr el programa, usar git...) sin salir de OpenCode. Además, OpenCode **recuerda el historial** de la sesión: por eso puede continuar tareas largas. Pero ese "recuerdo" es **contexto** y el contexto **gasta tokens**.

4.4 Ahorrar tokens y reducir el porcentaje usado

- Abre una **sesión nueva** (`/new`) al cambiar de tema.
- Sé **específico y breve** en tus peticiones.
- Usa **Plan** para no generar código a ciegas.
- Observa el **porcentaje de contexto usado** que muestra la TUI.

4.5 Flujo de un proyecto real

OpenCode permite pasar de una idea a una **app que funciona**:

- **SPEC.md** : describe *qué* quieres construir; OpenCode lo usa como guía (lo vemos en 4.6).
- **Tareas agrupadas**: divide el trabajo en pasos y los va completando.
- **Timeline**: revisar el avance y los cambios realizados.
- **Revertir** (`/undo`): volver a un punto anterior si algo sale mal.

4.6 El archivo `SPEC.md` (especificación)

Un `SPEC.md` describe **QUÉ** quieres construir, con detalle, **antes** de programar. No es una función oficial de OpenCode: es una **buena práctica** (desarrollo guiado por especificación). Le entregas el archivo al asistente para que tenga claro el objetivo y no se invente cosas.

Cómo usarlo (flujo recomendado)

1. Escribe tu `SPEC.md` con el objetivo, las funcionalidades y los criterios de aceptación.
2. En **modo Plan**, dile: `Lee @SPEC.md y propón un plan para construirlo`.
3. Revisa el plan; si te gusta, cambia a **modo Build** y dile que lo ejecute paso a paso.
4. Verifica cada **criterio de aceptación** del SPEC.

Un buen `SPEC.md` incluye: objetivo, usuarios, funcionalidades, qué queda fuera de alcance, requisitos técnicos y criterios de aceptación.

EJEMPLO LISTO PARA COPIAR

Tienes un `SPEC.md` de ejemplo completo en `clase-0/ejemplos-opencode/SPEC.md`.

4.7 Personalización

- **Tema y notificaciones:** cambiar colores de la TUI y recibir avisos al terminar tareas.
- **AGENTS.md**: archivo con reglas para el agente. OpenCode lo lee siempre; es como el "manual de la casa" (lo creamos en 4.8).
- **Comandos propios:** atajos para tareas repetidas, que pueden recibir **argumentos**.
- **Agentes y Skills:** sub-agentes especializados y "habilidades" empaquetadas que amplían lo que OpenCode sabe hacer.

4.8 Crear un archivo AGENTS.md (paso a paso)

El **AGENTS.md** es un archivo de texto con **instrucciones para el asistente**. OpenCode lo lee automáticamente en cada sesión, así no tienes que repetir las reglas. Es el "manual de la casa" de tu proyecto.

Dónde va (hay tres niveles)

Ubicación	Alcance
AGENTS.md en la raíz del proyecto	Solo ese proyecto (lo más común)
<code>~/.config/opencode/AGENTS.md</code>	Global: todos tus proyectos
<code>~/.claude/CLAUDE.md</code>	Compatibilidad (si vienes de Claude Code)

OpenCode busca en ese orden y usa el primero que encuentra.

Dos formas de crearlo

1. **Automática (recomendada):** dentro de OpenCode ejecuta `/init`. Revisa tu proyecto, a veces hace un par de preguntas y crea (o actualiza) el **AGENTS.md**.
2. **Manual:** crea el archivo **AGENTS.md** en la raíz y escríbelo tú mismo.

Qué escribir dentro

- Descripción del proyecto y tecnologías que usa.
- Estructura de carpetas.
- Cómo ejecutar y probar (los comandos exactos).
- Convenciones de código (idioma, estilo, nombres).
- Qué debe evitar el asistente.

EJEMPLO LISTO PARA COPIAR

Tienes un `AGENTS.md` de ejemplo completo y comentado en `clase-0/ejemplos-opencode/AGENTS.md`.

NOTA TÉCNICA

OpenCode **no** lee automáticamente otros archivos que menciones dentro del `AGENTS.md`. Si quieres incluir más archivos de reglas, indícalos en el campo `instructions` del archivo de configuración `opencode.json`.

4.9 Sesiones, terminal y modelos

- **Cambiar y compartir sesiones:** mantener varias y generar enlaces con `/share`.
- **Desde la terminal:** lanzar tareas directas y hasta levantar un servidor.
- **Modelos y pagos:** funciona con proveedores gratuitos y de pago; puedes conectar tu propia suscripción y comparar modelos por calidad, velocidad y costo.

CUIDADO AL COMPARTIR

Al usar `/share` se publica el contenido de la sesión en internet. No compartas sesiones con claves o datos privados.

4.10 Crear comandos, agentes y skills (con ejemplos)

OpenCode se puede personalizar con tres tipos de archivos. Todos se pueden guardar por proyecto (en la carpeta oculta `.opencode/`) o de forma global (en `~/.config/opencode/`).

a) Comando propio (a fondo)

Un comando es un **atajo para una tarea repetida**. Es un archivo `.md` y el **nombre del archivo es el comando** (por ejemplo, `componente.md` crea `/componente`).

Ejemplo básico (con argumentos):

```
# archivo: componente.md
---
description: Crea un componente de React
---
Crea un componente de React llamado $ARGUMENTS con TypeScript,
tipado correcto y una estructura básica.
```

Se usa escribiendo `/componente Boton . $ARGUMENTS` recibe todo lo que escribas después del comando; también puedes usar `$1`, `$2` ... por posición.

Dónde guardarlo — local, global o remoto:

Tipo	Ubicación	Alcance
Local (del proyecto)	<code>.opencode/commands/</code>	Solo ese proyecto
Global (de tu usuario)	<code>~/.config/opencode/commands/</code>	Todos tus proyectos de esa máquina
En configuración	Campo <code>command</code> en <code>opencode.json</code>	Según dónde esté el archivo

¿Y "remoto"? OpenCode no descarga comandos desde una URL. La forma de tener comandos "remotos" (compartidos con el equipo) es **subir la carpeta `.opencode/` a un repositorio Git** (por ejemplo GitHub): cuando otra persona **clona** el repositorio, recibe automáticamente todos los comandos del proyecto. Así, "local" = tu máquina, y "remoto/compartido" = lo que vive en el repositorio del equipo.

Comandos avanzados — puedes incluir archivos con `@` y meter la salida de la terminal con `!`...``:

```
# archivo: revisar-tests.md → /revisar-tests
---
description: Ejecuta las pruebas y sugiere mejoras
---
Estos son los resultados de las pruebas:
!`npm test`

Revisa también el componente @src/components/Boton.tsx
y, según los resultados, sugiere mejoras.
```

Otros campos útiles del frontmatter: `agent` (qué agente lo ejecuta), `model` (forzar un modelo) y `subtask` (ejecutarlo como subtarea).

b) Agente propio

Un asistente especializado. La forma fácil es el asistente interactivo:

```
opencode agent create
```

O a mano en `.opencode/agents/` (el nombre del archivo es el agente):

```
# archivo: .opencode/agents/documentador.md
---
description: Escribe y mantiene la documentación del proyecto
mode: subagent
permission:
  bash: deny
---
Eres un redactor técnico. Escribe documentación clara,
bien estructurada y con ejemplos de código.
```

El campo `mode` puede ser `primary` (lo eliges con `Tab`, como Build/Plan) o `subagent` (lo llama otro agente para una subtarea).

c) Skill (habilidad)

Una habilidad empaquetada que el agente activa **solo** cuando encaja con lo que pides. Va en `.opencode/skills/<nombre>/SKILL.md` con frontmatter obligatorio (`name` y `description`).

```
# archivo: .opencode/skills/notas-version/SKILL.md
---
name: notas-version
description: Crea notas de versión y changelog consistentes
---
## Qué hago
- Redacto notas de versión a partir de los PR combinados
- Propongo el nuevo número de versión
- Doy un comando `gh release create` listo para copiar

## Cuando usarme
Cuando estés preparando una nueva versión etiquetada.
```

4.11 Conectar tu suscripción y pagar OpenCode (Zen)

Conectar una suscripción existente (por ejemplo, ChatGPT)

Además de una API key, puedes iniciar sesión con una suscripción:

1. En OpenCode escribe `/connect`.
2. Elige el proveedor **OpenAI** y la opción "**ChatGPT Plus/Pro**".
3. Se abre el navegador para **iniciar sesión** y autorizar.

También soporta iniciar sesión con **GitHub Copilot** y **GitLab Duo**. Las credenciales quedan guardadas en `~/.local/share/opencode/auth.json`.

IMPORTANTE

Anthropic **no permite** usar la suscripción Claude Pro/Max en herramientas de terceros. Para usar Claude en OpenCode, usa una **API key**.

Pagar OpenCode (OpenCode Zen)

OpenCode Zen es el servicio del equipo de OpenCode: una lista de modelos probados, con **pago por uso** (se cobra por tokens) y también algunos modelos **gratuitos**.

1. Inicia sesión en **OpenCode Zen** y agrega tus datos de **facturación** (tarjeta).
2. Obtén tu **API key** de Zen.
3. En OpenCode: `/connect` → elige **OpenCode Zen** → pega la key.
4. Selecciona el modelo con el formato `opencode/<modelo>` (por ejemplo `opencode/gpt-5.5`).

PARA EL CURSO

No necesitas pagar nada: con las APIs **gratuitas** (Gemini y Cohere) es suficiente. OpenCode Zen es opcional para cuando quieras modelos más potentes.

4.12 Buenas prácticas con OpenCode

- **Empieza en Plan, luego Build.** Acuerda el enfoque antes de que toque el código (`Tab` cambia de modo).
- **Usa un `AGENTS.md`** desde el inicio con `/init` : menos repetir, mejores resultados.
- **Escribe un `SPEC.md`** para proyectos con varias partes.
- **Sé específico.** "Crea una función que valide un correo con expresión regular" es mejor que "arréglame esto".
- **Da contexto con `@`** en vez de copiar y pegar archivos.
- **Trabaja en pasos pequeños** y revisa cada cambio; usa `/undo` si algo sale mal.
- **Nueva sesión (`/new`) al cambiar de tema** y `/compact` en charlas largas: ahorra tokens.
- **Usa control de versiones (Git).** Haz commits frecuentes para poder volver atrás.
- **Nunca subas claves.** Mantén el `.env` en `.gitignore` .
- **No confíes a ciegas.** Lee y prueba lo que genera: la IA se puede equivocar con seguridad.

4.13 Video-tutorial (de básico a avanzado)

Para verlo paso a paso, sigue este tutorial y usa las marcas de tiempo:

<https://www.youtube.com/watch?v=ZZq4TpNgnvg>

Tema	Min.	Tema	Min.
¿Qué es OpenCode?	0:00	Timeline / revertir	30:50
Instalación + TUI	2:58	Cambiar el tema	32:50
Ventajas	5:30	Notificaciones / revertir	34:19
Primer prompt	9:32	Modelos de pago (GO)	38:51
Comandos y TUI	11:22	App que funciona	43:05
Build y Plan	13:57	AGENTS.md	52:04
Ahorrar tokens	16:53	Cambiar de sesión	1:01:35
Modo Shell	20:10	Skills (AgentSkill)	1:05:32
Importancia del historial	22:08	Conectar suscripción	1:25:00
Proyecto desde cero	23:25	Comparar modelos	1:33:01
SPEC.md	27:58	Agentes	1:35:14
Tareas agrupadas	29:39	Comandos propios / args	1:42:10

5. Práctica guiada: "Hola IA"

```
pip install google-generativeai python-dotenv
```

Crea `hola_ia.py` (ya lo tienes en `clase-0/codigo/`) y ejecútalo con `python hola_ia.py`. Debe responderte el modelo.

LABORATORIO INTERACTIVO DEL CURSO

A lo largo del curso usaremos demos en vivo para **ver** cómo funciona la IA (cómo un LLM predice el siguiente token, cómo aprende una red neuronal, etc.). Están reunidas en el hub: recursos/demos-interactivos.html. Puedes ir explorándolas desde ya.

Actividades de refuerzo

Realiza estas actividades para afianzar lo aprendido. Puedes escribir tus respuestas en los espacios en blanco.

1 Con mis palabras Fácil

Explica con una analogía propia (distinta al restaurante) qué es una API y qué es una API key.

2 Verificación del entorno Fácil

Ejecuta los siguientes comandos y anota la versión que te muestra cada uno:

Comando	Resultado (versión)
<code>python --version</code>	
<code>opencode --version</code>	
<code>node --version</code>	

3 Tres preguntas a la IA Media

Usando OpenCode o `hola_ia.py`, hazle 3 preguntas distintas al modelo. Anota la que más te haya sorprendido y por qué.

4 Crea con OpenCode Media

Pídele a OpenCode que cree un programa que convierta grados Celsius a Fahrenheit. Ejecútalo y verifica que funcione. Anota qué instrucción exacta le diste.

5 Plan antes que Build Media

En OpenCode, cambia al modo **Plan** y pídele que planee una calculadora sencilla. Anota los pasos que propone. Luego pasa a **Build** y deja que la construya. ¿En qué se diferenció el resultado de cada modo?

6 Mi primera app con OpenCode Reto

Crea una carpeta nueva y, con OpenCode, construye una **página web sencilla** (por ejemplo, una tarjeta de presentación con tu nombre en HTML y CSS). Ábrela en el navegador. Si algo sale mal, usa `/undo` para revertir. Anota qué le pediste y qué comando usaste para revertir.

7 Explora el video Fácil

Mira al menos **3 secciones** del video-tutorial de OpenCode (usa las marcas de tiempo de la sección 4.8) y escribe una función de OpenCode que no conocías y para qué sirve.

8 Detective de IA Reto

Escribe 5 situaciones de tu vida diaria donde ya usas IA sin darte cuenta (por ejemplo, recomendaciones de video, autocorrector, etc.) y explica brevemente qué crees que hace la IA en cada una.

Lista de verificación final

- ☐ Tengo mi API key de Gemini y la probé.
- ☐ Tengo mi API key de Cohere.
- ☐ Tengo cuenta de GitHub.
- ☐ Python y VS Code funcionan.
- ☐ OpenCode instalado y me creó un archivo.
- ☐ Mi archivo `.env` está guardado y sé que es secreto.

SOLUCIÓN DE PROBLEMAS

python no se reconoce: reinstala Python marcando "Add to PATH". · **pip no se reconoce:** usa `python -m pip install ...` · **API key inválida:** revisa que se copió completa, sin espacios. · **Se acabó el cupo:** cambia de proveedor (Cohere / GitHub Models).